

Architettura degli Elaboratori

L'architettura della CPU – Gestione dei data hazard



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo

Masi, Claudio di Ciccio

[S&PdC]

4.4 - 4.7



Scoprire un data-hazard in EXE

Legenda: Segnali dei registri di pipeline: `<reg.pipeline>.<segnale>`

- 1.a) `ID/EX.rs1 == EX/MEM.rd` `&& EX/MEM.RegWrite == 1` `&& EX/MEM.rd != 0`
- 1.b) `ID/EX.rs2 == EX/MEM.rd` `&& EX/MEM.RegWrite == 1` `&& EX/MEM.rd != 0`
- 2.a) `ID/EX.rs1 == MEM/WB.rd` `&& MEM/WB.RegWrite == 1` `&& MEM/WB.rd != 0`
- 2.b) `ID/EX.rs2 == MEM/WB.rd` `&& MEM/WB.RegWrite == 1` `&& MEM/WB.rd != 0`

- ① `sub x2, x1, x3 #` IF | ID | EX | MM | WB
- ② `and x12, x2, x5 #` IF | ID | EX | MM | WB (1.a)
- ③ `or x13, x6, x2 #` IF | ID | EX | MM | WB (2.b)
- ④ `add x14, x2, x2 #` IF | ID | EX | MM | WB
- ⑤ `sw x15, 100(x2) #` IF | ID | EX | MM | WB

Se è presente un **data-hazard** su un argomento rispetto a **entrambe** le istruzioni precedenti?

Ha sempre la precedenza il valore prodotto per ultimo, ossia il contenuto di EX/MEM della pipeline (dell'istr. subito precedente a quella corrente in EXE)

Scoprire un data-hazard in EXE

Legenda: Segnali dei registri di pipeline: <reg.pipeline>.<segnale>

1.a) ID/EX.rs1 == EX/MEM.rd && EX/MEM.RegWrite == 1 && EX/MEM.rd != 0
1.b) ID/EX.rs2 == EX/MEM.rd && EX/MEM.RegWrite == 1 && EX/MEM.rd != 0
2.a) ID/EX.rs1 == MEM/WB.rd && MEM/WB.RegWrite == 1 && MEM/WB.rd != 0
2.b) ID/EX.rs2 == MEM/WB.rd && MEM/WB.RegWrite == 1 && MEM/WB.rd != 0

①	sub	x2, x1, x3	#	IF		ID		EX		MM		WB	
②	and	x2, x2, x5	#			IF		ID		EX		MM	WB (1.a)
③	or	x13, x6, x2	#					IF		ID		EX	MM WB (1.b)
④	add	x14, x2, x2	#							IF		ID	EX MM WB (2.a, b)
⑤	sw	x15, 100(x2)	#									IF	ID EX MM WB

Se è presente un **data-hazard** su un argomento rispetto a **entrambe** le istruzioni precedenti?

Ha sempre la precedenza il valore prodotto per ultimo, ossia il contenuto di EX/MEM della pipeline (dell'istr. subito precedente a quella corrente in EXE)

Precedenza fra i data-hazard in EXE

Legenda: Segnali dei registri di pipeline: <reg.pipeline>.<segnale>

①	add	x1,x1,x2	#	IF		ID		EX		MM		WB				
②	add	x1,x1,x3	#			IF		ID		EX		MM		WB		
③	add	x1,x1,x4	#					IF		ID		EX		MM		WB

```
if    (MEM/WB.RegWrite
and   (MEM/WB.RegistroRd ≠ 0)
and   not(EX/MEM.RegWrite and (EX/MEM.RegistroRd ≠ 0)
        and (EX/MEM.RegistroRd = ID/EX.RegistroRs1))
and   (MEM/WB.RegistroRd = ID/EX.RegistroRs1)) PropagaA = 01
```

Propaga MEM/WB solo se

- 1) necessario ($ID/EX.rs1 == MEM/WB.rd$) e
 - 2) EX/MEM non è attivo (v. righe in blu)
- (la stessa aggiunta va fatta al test per rs2)

Se è presente un **data-hazard** su un argomento rispetto a **entrambe** le istruzioni precedenti?

Ha sempre la precedenza il valore prodotto per ultimo, ossia il contenuto di EX/MEM della pipeline (dell'istr. subito precedente a quella corrente in EXE)

Perché controllare $rd \neq 0$

- Vedremo che ci sono speciali operazioni (nop) in cui si scrive (fittiziamente) su x0
- Questo è un caso speciale che non è un vero data hazard!
- In generale se sto scrivendo su x0 non ha senso considerarlo un data hazard

(nop =) addi x0, x0, 0

Realizzare il forwarding in EXE

Forwarding: sostituire il valore letto dal blocco registri con quello prodotto dall'istruzione precedente in fase EXE (o dalla 2° istruzione precedente in fase MEM)

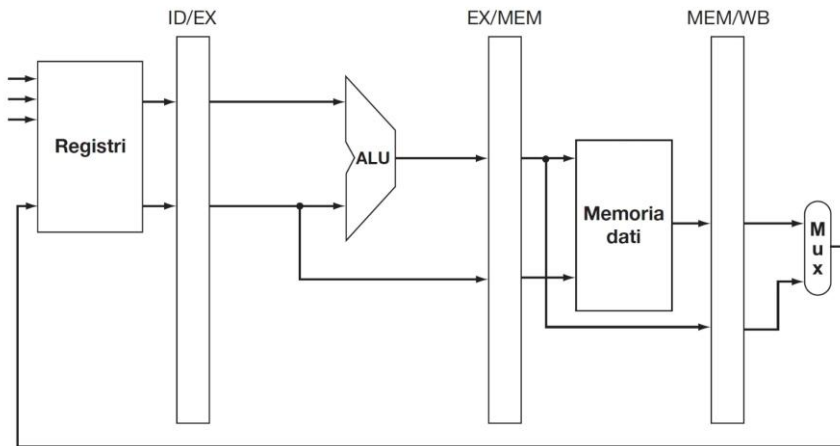
Modifiche al datapath: inserire un **MUX** prima della ALU per selezionare tra i 3 casi:

- **Non c'è forwarding:** il valore per la ALU viene dal registro **ID/EX** della pipeline
- **Forwarding dall'istr. prec.:** il valore per la ALU viene dal registro **EX/MEM** della pipel.
- **Forwarding dalla 2° istr. prec.:** il valore per la ALU viene dal registro **MEM/WB** della pipel.

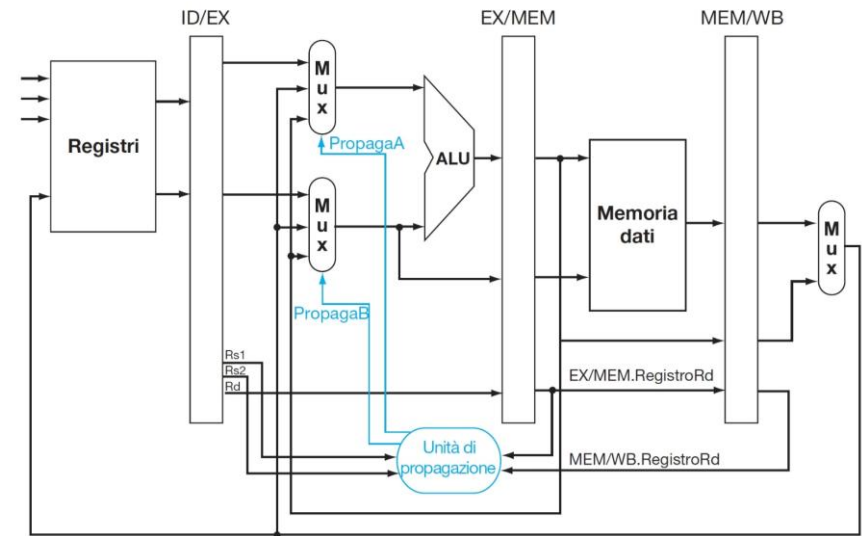
Questo vale sia per il primo argomento che per il secondo argomento della ALU (**2 MUX**)

Unità di forwarding

Senza forwarding (schema semplificato)



Con forwarding (schema semplificato)



Segnali per il forwarding

Controllo multiplexer	Sorgente	Spiegazione
PropagaA = 00	ID/EX	Il primo operando della ALU proviene dal register file
PropagaA = 10	EX/MEM	Il primo operando della ALU viene propagato dal risultato della ALU nel ciclo di clock precedente
PropagaA = 01	MEM/WB	Il primo operando della ALU viene propagato dalla memoria dati o da un precedente risultato della ALU
PropagaB = 00	ID/EX	Il secondo operando della ALU proviene dal register file
PropagaB = 10	EX/MEM	Il secondo operando della ALU viene propagato dal risultato della ALU nel ciclo di clock precedente
PropagaB = 01	MEM/WB	Il secondo operando della ALU è propagato dalla memoria dati o da un precedente risultato della ALU

Realizzare il forwarding in EXE

Forwarding: sostituire il valore letto dal blocco registri con quello prodotto dall'istruzione precedente in fase EXE (o dalla 2° istruzione precedente in fase MEM)

Modifiche al datapath: inserire un **MUX** prima della ALU per selezionare tra i 3 casi:

- **Non c'è forwarding:** il valore per la ALU viene dal registro **ID/EX** della pipeline
- **Forwarding dall'istr. prec.:** il valore per la ALU viene dal registro **EX/MEM** della pipel.
- **Forwarding dalla 2° istr. prec.:** il valore per la ALU viene dal registro **MEM/WB** della pipel.

Questo vale sia per il primo argomento che per il secondo argomento della ALU (**2 MUX**)

NOTA: è possibile realizzare il forwarding anche (ma non lo introdurremo):

MEM -> MEM (necessario SOLO se **lw rs1, ...** è subito seguita da **sw rs1, ...**)
(se sw è preceduta da tipo R il forwarding avviene in fase EXE)

Data hazard di lw seguita da una sw

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

Data hazard di lw seguita da una sw

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

Quando vogliamo spostare un valore **dalla** memoria e posizionarlo in un **altro valore della memoria**. Una sorta di swap di valori in memoria.

$\text{Mem}[\text{offset}+\text{t3}] \leftarrow \text{Mem}[\text{offset}+\text{t1}]$

```
lw  t0, offset(t1)
```

```
sw  t0, offset(t3)
```

Data hazard di lw seguita da una sw

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

Quando vogliamo spostare un valore **dalla** memoria e posizionarlo in un **altro valore della memoria**. Una sorta di swap di valori in memoria.

$\text{Mem}[\text{offset}+\text{t3}] \leftarrow \text{Mem}[\text{offset}+\text{t1}]$

lw **t0**, **offset(t1)** **IF** | **ID** | **EX** | **MM** | **WB**

sw **t0**, **offset(t3)** **IF** | **ID** | **EX** | **MM** | **WB**

Data hazard di lw seguita da una sw

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

Quando vogliamo spostare un valore **dalla** memoria e posizionarlo in un **altro valore della memoria**. Una sorta di swap di valori in memoria.

$\text{Mem}[\text{offset}+\text{t3}] \leftarrow \text{Mem}[\text{offset}+\text{t1}]$

lw	t0 , offset(t1)	IF	ID	EX	MM	WB			
sw	t0 , offset(t3)		->	->	IF	ID	EX	MM	WB

senza forwarding
sono necessari due stalli

Data hazard di lw seguita da una sw

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

Quando vogliamo spostare un valore **dalla** memoria e posizionarlo in un **altro valore della memoria**. Una sorta di swap di valori in memoria.

$\text{Mem}[\text{offset}+\text{t3}] \leftarrow \text{Mem}[\text{offset}+\text{t1}]$

lw	t0	, offset(t1)	IF	ID	EX	MM	WB	
sw	t0	, offset(t3)	->	IF	ID	EX	MM	WB

con l'unità di forwarding
è necessario uno stallo
(forwarding MM->EX)

Realizzare il forwarding in MEM (lw/sw)

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

Quando vogliamo spostare un valore **dalla** memoria e posizionarlo in un **altro valore della memoria**. Una sorta di swap di valori in memoria.

$\text{Mem}[\text{offset}+t3] \leftarrow \text{Mem}[\text{offset}+t1]$

lw	t0	, offset(t1)	IF	ID	EX	MM	WB
sw	t0	, offset(t3)		ID	EX	MM	WB

Il dato in t0 è
prodotto qui



Realizzare il forwarding in MEM (lw/sw)

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

Quando vogliamo spostare un valore **dalla** memoria e posizionarlo in un **altro valore della memoria**. Una sorta di swap di valori in memoria.

$\text{Mem}[\text{offset}+\text{t3}] \leftarrow \text{Mem}[\text{offset}+\text{t1}]$

lw	t0	, offset(t1)	IF	ID	EX	MM	WB	
sw	t0	, offset(t3)		IF	ID	EX	MM	WB

Il dato in t0 è
prodotto qui

sw lo scrive
qua in memoria

Realizzare il forwarding in MEM (lw/sw)

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

lw **t0**, **offset(t1)**

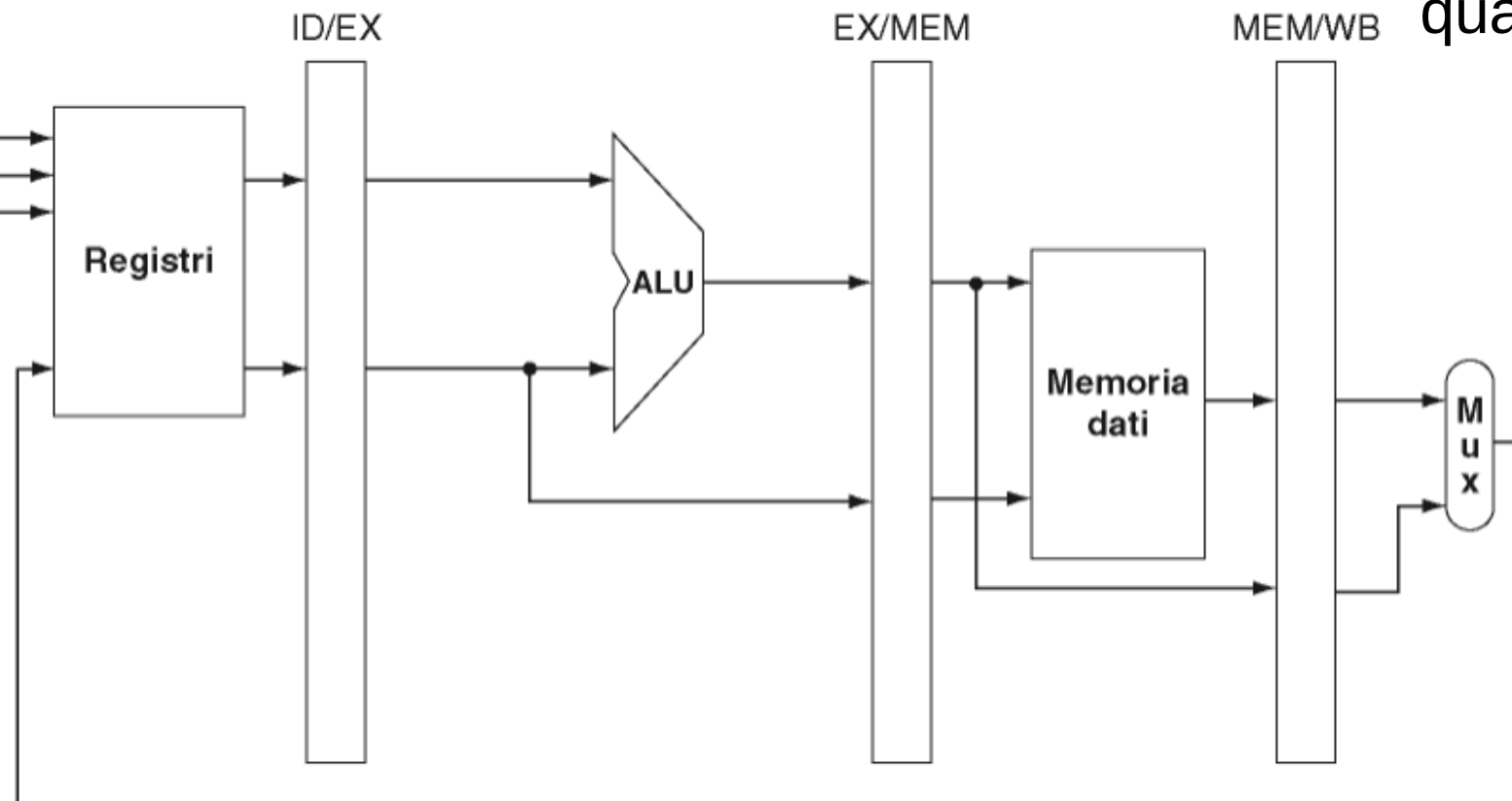
IF | ID | EX | **MM** | **WB**

sw **t0**, **offset(t3)**

IF | ID | **EX** | **MM** | WB

Il dato in t0 è
prodotto qui

sw lo scrive
qua in memoria



Realizzare il forwarding in MEM (lw/sw) – possibile soluzione

In quali casi viene usato **lw** e poi **sw** in questo esatto ordine?

lw **t0**, **offset(t1)**

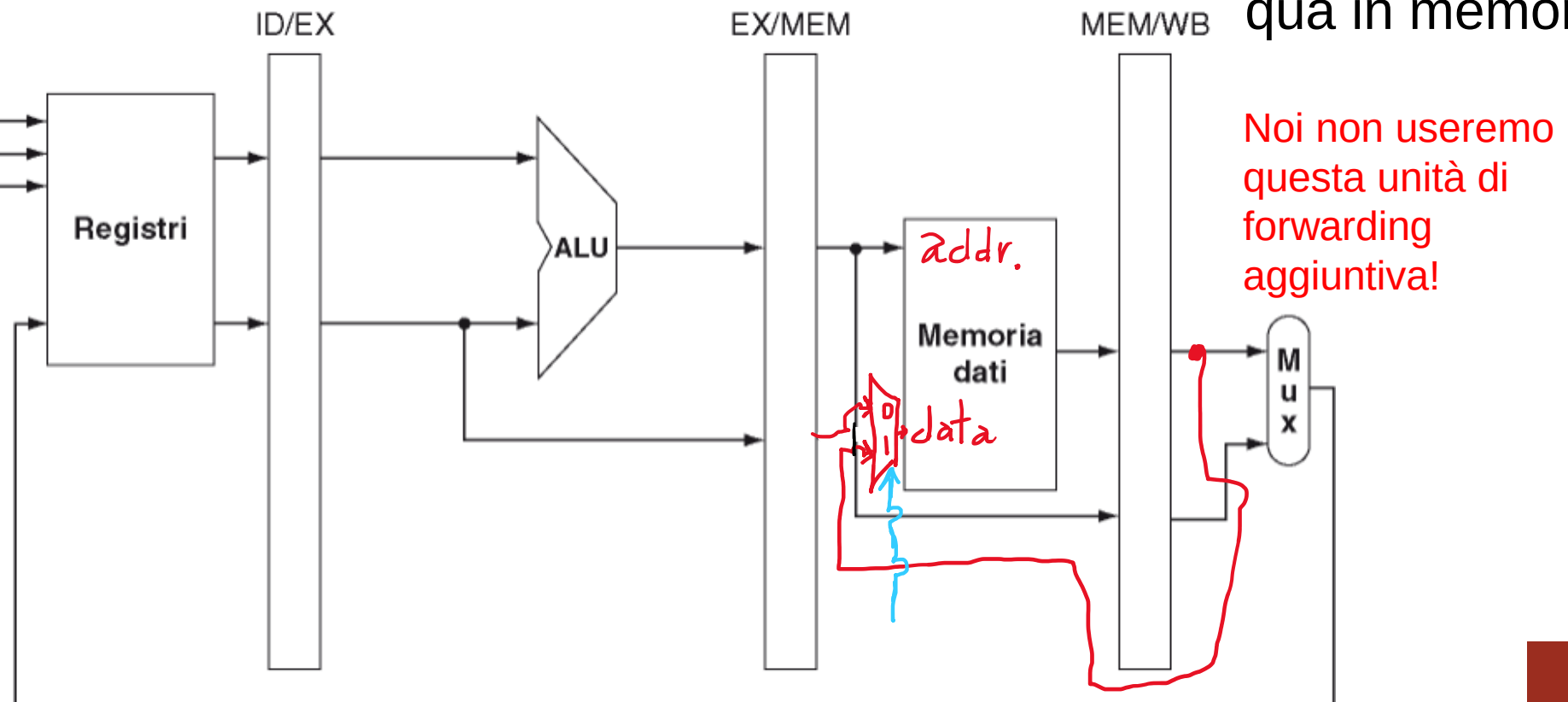
IF | ID | EX | **MM** | **WB**

sw **t0**, **offset(t3)**

IF | ID | **EX** | **MM** | WB

Il dato in t0 è
prodotto qui

sw lo scrive
qua in memoria



Come rilevare il forwarding?

Un possibile modo di rilevazione:

$\text{MEM/WB.MemToReg} == 1 \quad \&\& \quad \text{MEM/WB.RegWrite} == 1 \quad \&\& \quad \text{EX/MEM.MemWrite} == 1$
 $\&\& \text{MEM/WB.Rd} == \text{EX/MEM.Rs2}$

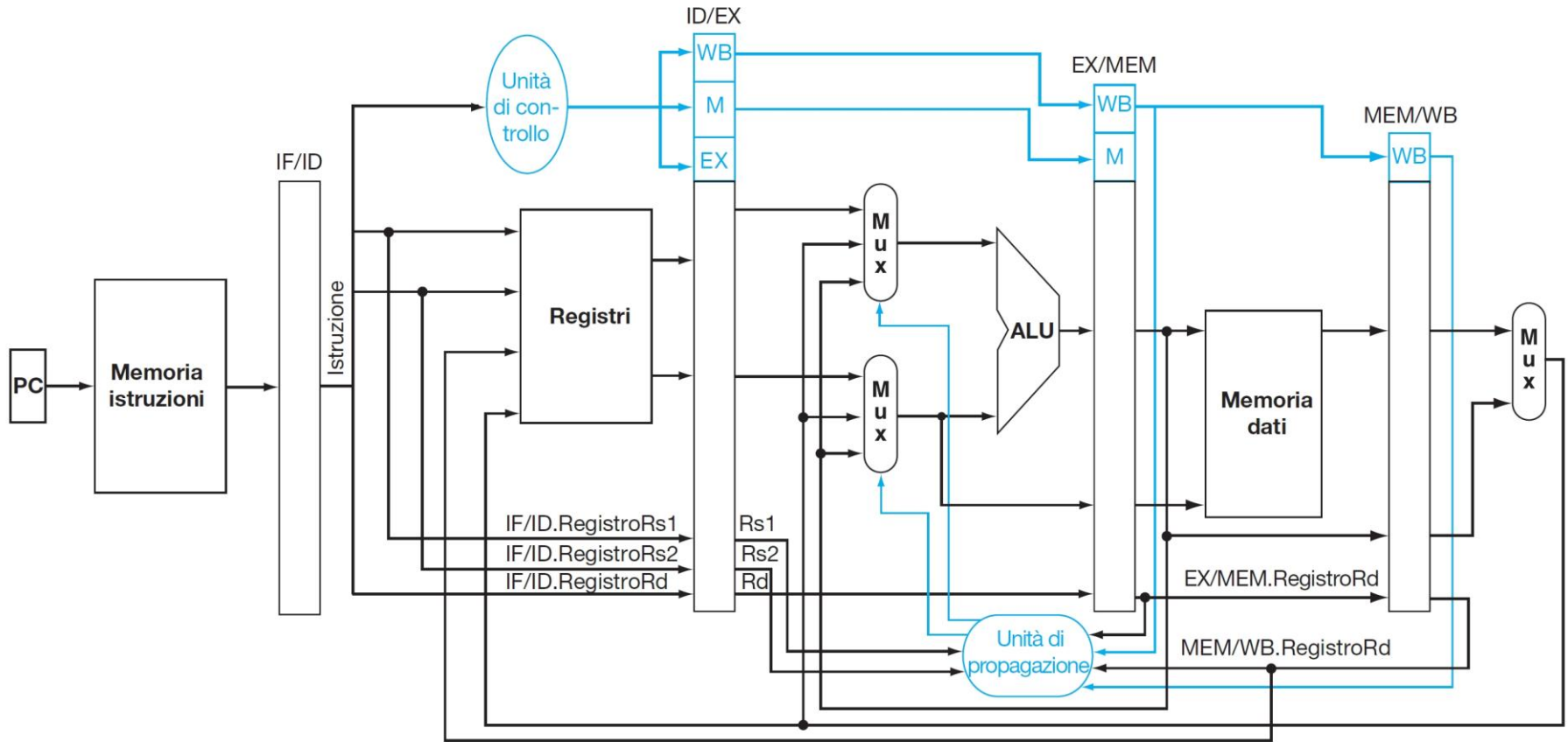
Razionale:

1. La `lw` deve scrivere in WB nel register file dalla memoria
2. Nello stadio a monte (istruzione successiva del codice) occorre scrivere in memoria (`sw`)
3. I registri sono uguali ($\text{MEM/WB.Rd} == \text{EX/MEM.Rs2}$)

Noi non useremo
questa unità di
forwarding
aggiuntiva!

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

CPU con forwarding per data hazard (senza stallo)



Lo stallo dell'istruzione

Talvolta l'istruzione deve attendere che sia pronto il dato perché possa avvenire il forwarding

- ① `lw s1, (s2)` # IF ID EX **MM** WB
- ② `addi s1, s1, 42` # **->** IF **ID** **EX** MM WB
- ③ `addi s1, s1, 42` # **(->)** IF ID **EX** MM WB

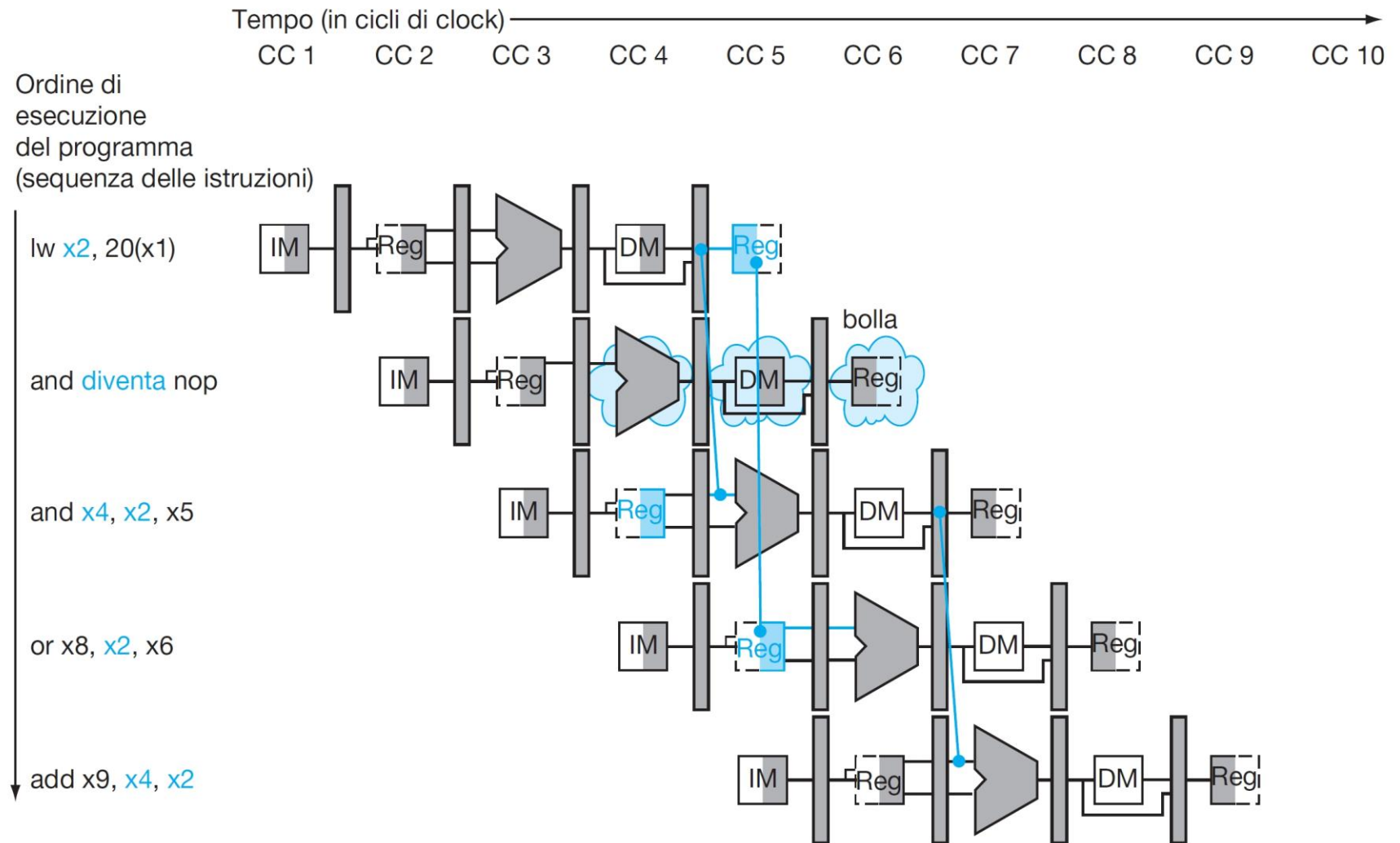
Per riconoscere di dover inserire uno stallo dobbiamo esaminare le due istruzioni:

- la seconda ne ha bisogno in EXE e la prima produrrà il dato solo nella fase MEM (**lw**)
`ID/EX.MemRead == 1 && (IF/ID . rs1 == ID/EX . rd || IF/ID . rs2 == ID/EX . rd)`
- la seconda lo richiede in ID e la prima lo produrrà solo dopo EXE (solo **beq** anticipato)
`ID/EX.RegWrite == 1 && (IF/ID . rs1 == ID/EX . rd || IF/ID . rs2 == ID/EX . rd) && op == beq`

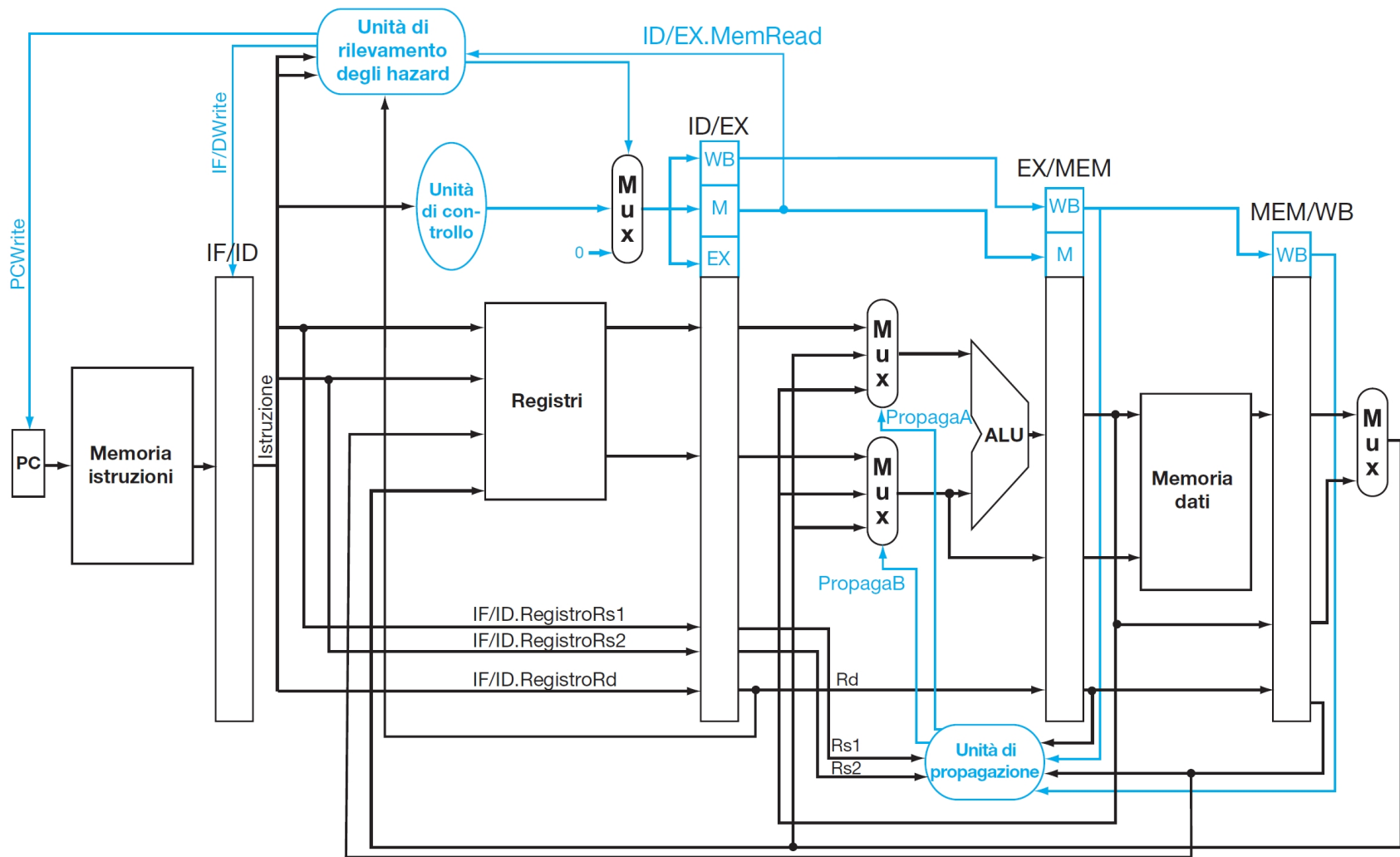
Per fermare l'istruzione con uno stallo, dobbiamo (nella fase ID):

- annullare l'istruzione che deve attendere (**bolla**)
 - occorre **azzerare i segnali di controllo MemWrite e RegWrite** nonché **IF/ID.Istruzione**
- **rileggere la stessa istruzione** di nuovo affinché possa essere rieseguita un CC dopo
 - occorre **impedire che il PC si aggiorni**

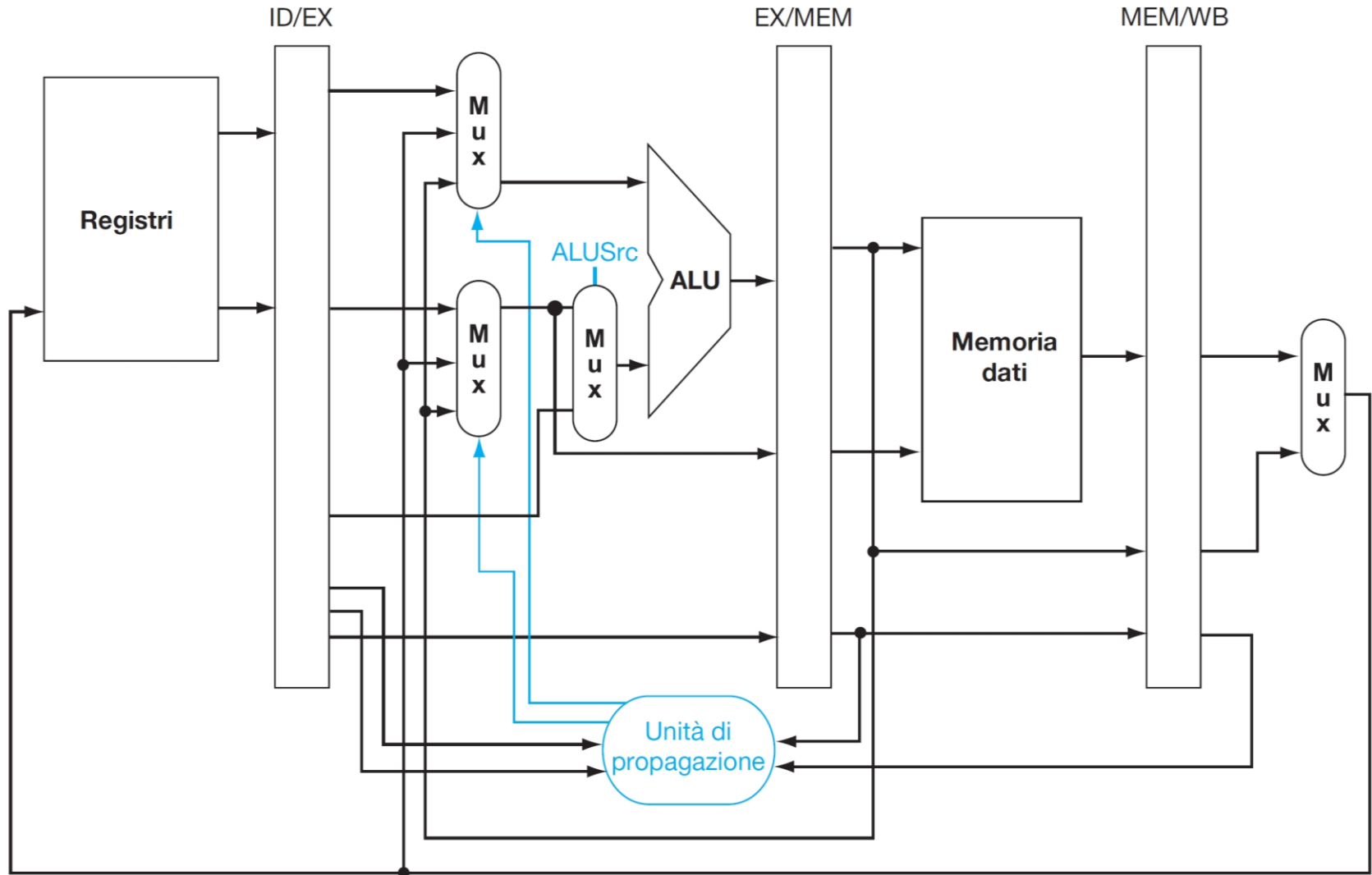
Stallo in azione



Architettura (quasi) completa



Con forwarding (schema semplificato)





Esercizio per casa: da **vj** a **vjal**

Si vuole aggiungere alla CPU l'istruzione vectorised jump (**vj**), di tipo I e sintassi assembly

vj *indice*, **x0**, *vettore*

che salta all'indirizzo contenuto nell'elemento indice-esimo del *vettore di word*. Il secondo registro (**rs2**) non è utilizzato e per questo viene settato a zero

L'assemblatore inserirà nella parte immediata, i 12 bit di scostamento relativi al program counter per raggiungere l'indirizzo di memoria dell'etichetta *vettore*. In altre parole, l'indirizzo di memoria dell'etichetta è uguale $PC + [20-31]$ (i 12 bit dell'istruzione).

Esempio: se in memoria si è definito staticamente il

vettore: **.word** 0x10008000, 0x10008200, 0x10001088, 0x10008444

e al registro **t0** è assegnato il valore **3** allora

vj **t0**, *vettore*

salterà all'indirizzo **0x10008444** (che è l'elemento con indice 3 del vettore).

- si disegnino le modifiche necessarie a realizzare la funzione, aggiungendo tutti gli eventuali MUX, segnali di controllo, bus, ALU e sommatori (ecc.) che ritenete necessari.
- indicate i valori di tutti i segnali di controllo, in modo da eseguire l'istruzione **vj**.
- tenendo conto che il tempo di accesso ai registri (sia in lettura che scrittura) è di **25ns**, l'accesso alla memoria impiega **75ns**, la ALU e i sommatori impiegano **100ns** e ignorando gli altri ritardi, calcolate il tempo di esecuzione minimo della istruzione **vj** e indicate se è necessario aumentare il periodo di clock della CPU per poter svolgere questa nuova istruzione.

Esercizio per casa: modificare **vj** in **vjal** (salvando il pc corrente in **ra** prima del salto)

Esercizi per casa



SAPIENZA
UNIVERSITÀ DI ROMA

Esercizio

Nota: Assumiamo che
la `s0`, etichetta
sia un'istruzione del set standard (equivalente
ad una `addi`). In realtà l'assemblatore
potrebbe doverla spezzare in due istruzioni

Esercizio (esame 16-9-14)

1. Individuare i **data** e **control hazard**

- assumendo la **decisione** dei
branch in fase ID

2. Individuare i casi in cui il **forwarding è applicabile**

3. Individuare gli **stalli necessari**

4. Calcolare la **durata totale** in cicli di clock **con forwarding**

5. Calcolare la **durata senza forwarding**

6. Riordinare le istruzioni per ridurre gli stalli (con fwd.)

7. Calcolare la **durata** del programma **ottimizzato (4.)**

8. Cosa c'è in **pipeline** al **16° CC** (v. programma originale)?

```
.data
```

```
vettore: .word 24, ..., 2, 11, 39, 4 # 7 pari, 3 dispari
```

```
.text
```

```
main:
```

```
    addi t0, zero, 0
```

```
    la    s0, vettore
```

```
    addi t1, s0, 36
```

```
ciclo:
```

```
    lw    t3, 0(t1)
```

```
    andi t2, t3, 1
```

```
    beq   t2, zero, salta
```

```
    add   t0, t0, t3
```

```
salta:
```

```
    addi t1, t1, -4
```

```
    bge   t1, s0, ciclo
```

```
    addi a7,x0, 1
```

```
    mv    a0, t0
```

```
    ecall
```

```
# Sommo solo i dispari
```

```
# somma parziale
```

```
# indirizzo base
```

```
# offset ultimo
```

```
# è dispari?
```

```
# se non lo è, salto
```

```
# altrimenti, lo sommo
```

```
# prossimo elemento
```

```
# fine del ciclo?
```

```
# stampa...
```

```
# ... la somma
```

